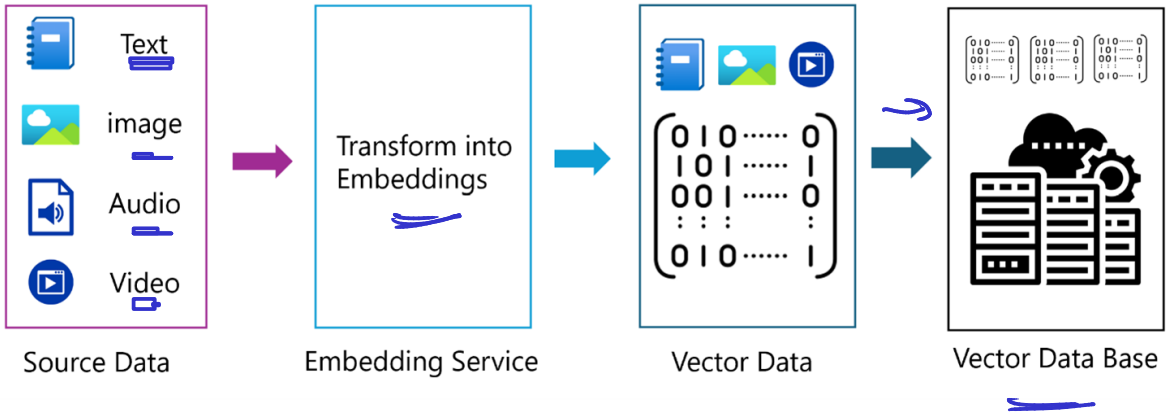
Vector DB



Why vector DB

Data → C1-Emb, C2-Emb, C3-Emb, Cn-Emb

DB → store information

SQL DB / No-SQL DB

Table

Chunk	Embedding
1.....	.....
2. ---	----
3 ---	...

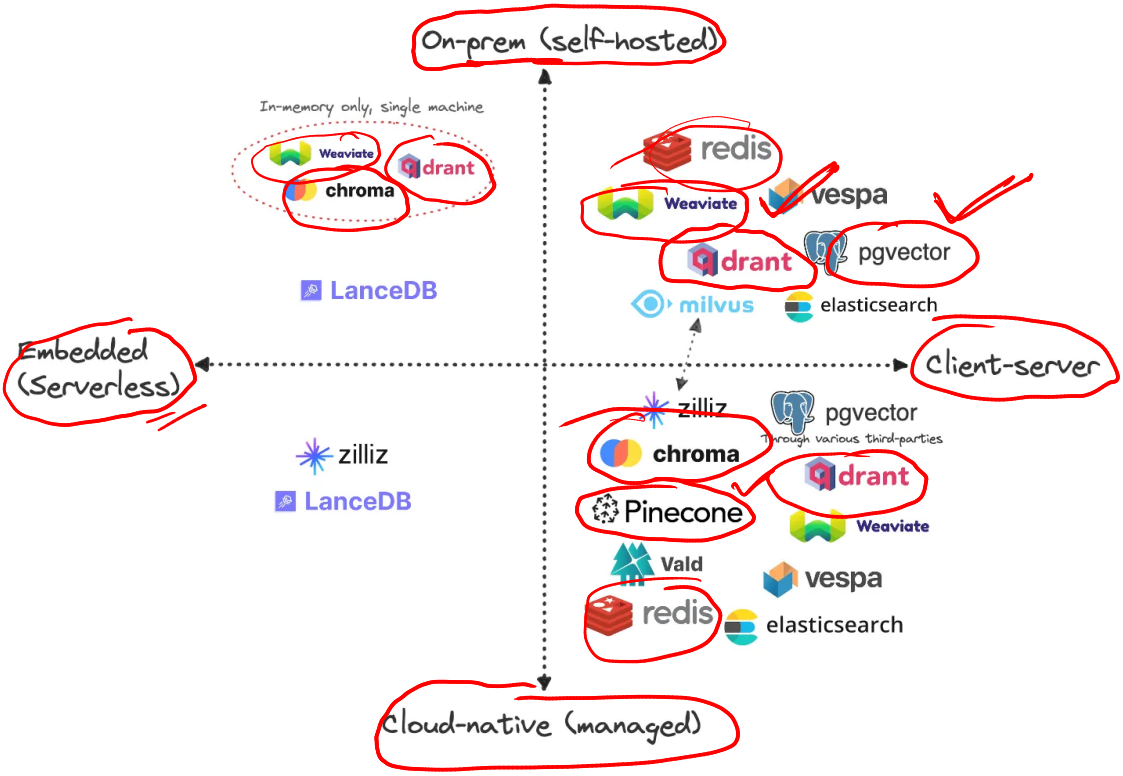
Question

Efficient

Vector DB → Similarity search

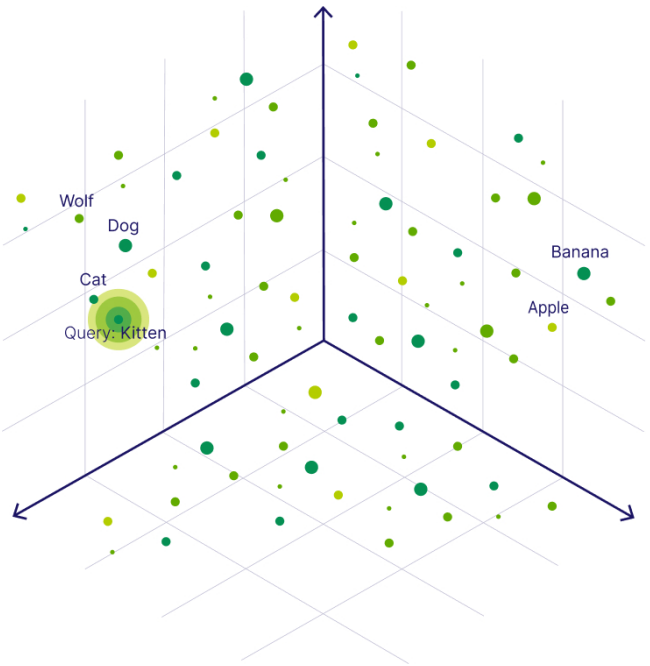
Indexing ⇒ way we store the vector

efficient → similarity search



Indexing Mechanism

Indexing



Indexing

Technique to store the Embedding into vector DB

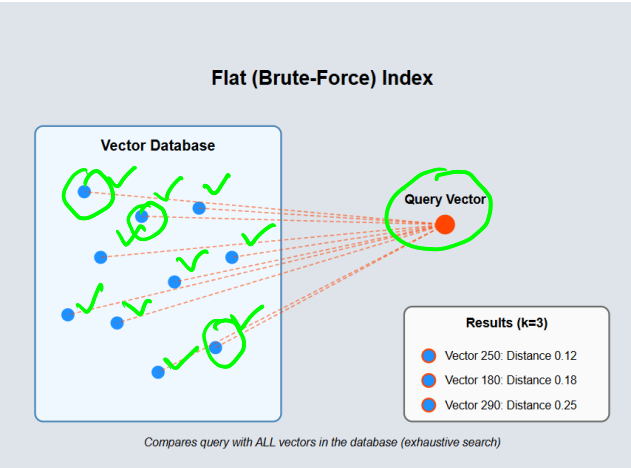
Organize the Embedding in a structred way

⇩

retrevial will be faster & efficient

Brute Force Search

This method stores vectors without any compression or special structure and calculates the distance between the query vector and every other vector in the database.



- Strategic Wins
  - Guarantees 100% accurate results (perfect recall).
  - No index training time required.
  - Simplest implementation.

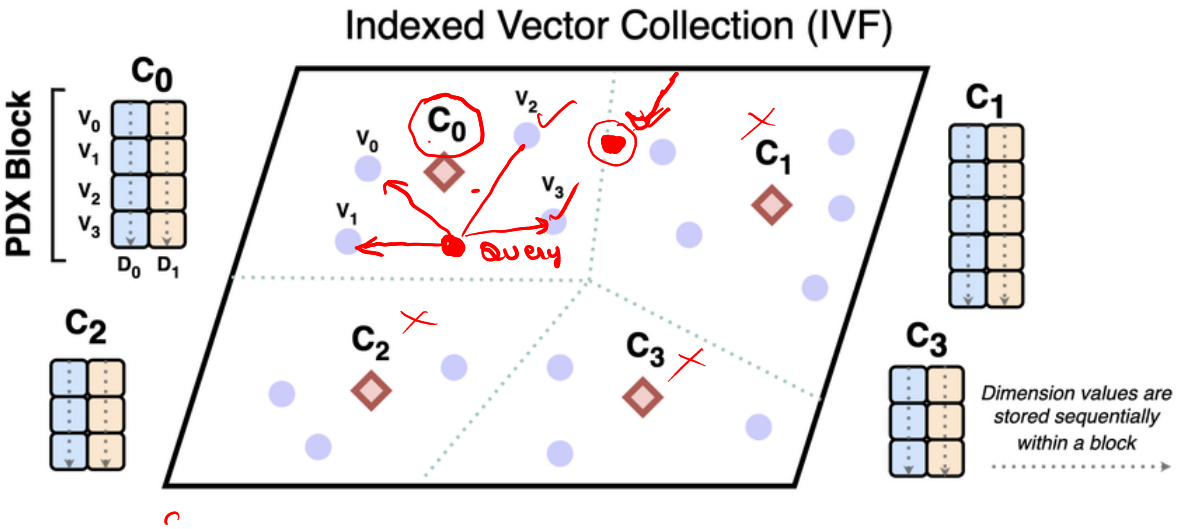
Critical Trade-offs

! Inefficient and slow for large datasets (O(n) complexity).

! Best suited for small, static datasets where accuracy is paramount.

- **Inverted File Index (IVF)**

This technique divides the dataset into clusters and identifies cluster centroids.
During a search, the system only compares the query vector to the vectors within the most relevant clusters.

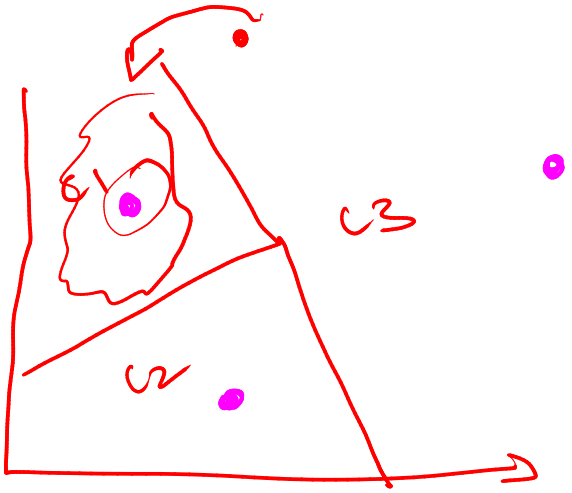


Query → Embedding

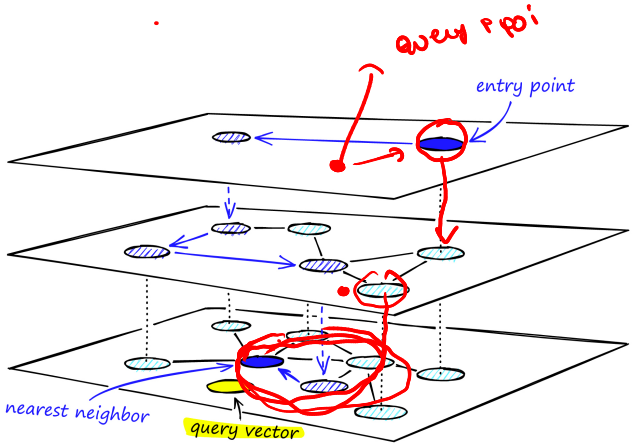
Reducing the search space

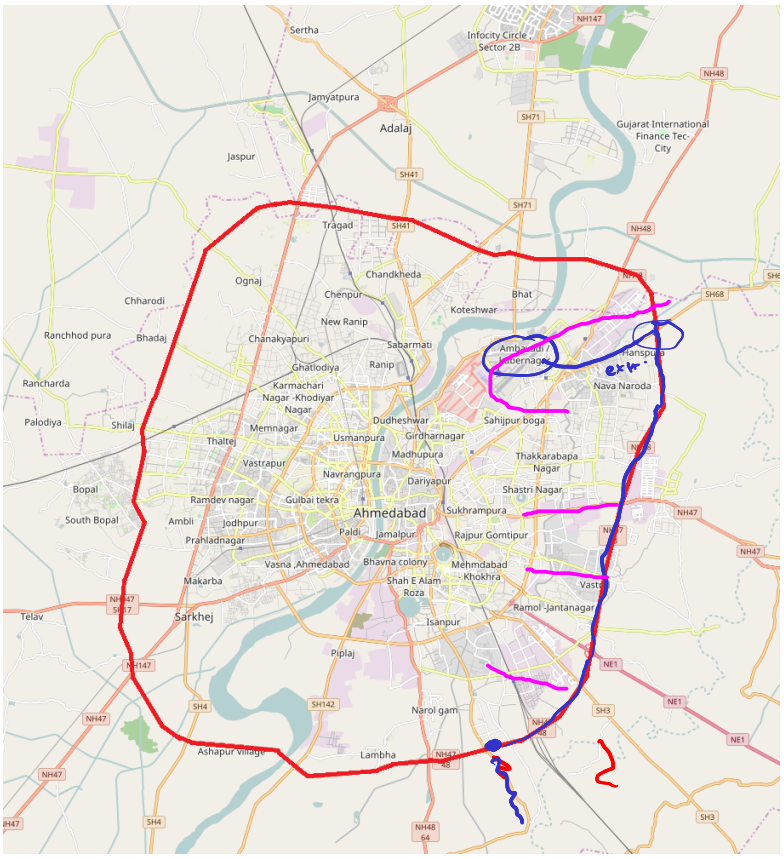
Strategic Wins
- Offers much faster search times than flat indexing.
- Scales well to large datasets.
- Relatively intuitive structure.

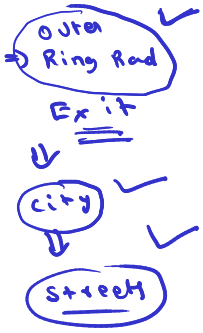
Critical Trade-offs
! Accuracy depends on the number of clusters (nlist) and clusters searched (nprobe).
! Requires an initial training phase to find centroids

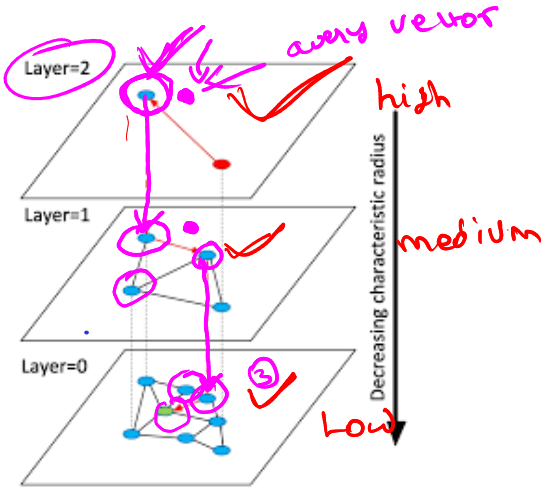


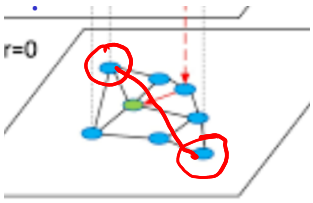
Hierarchical Navigable Small World (HNSW)

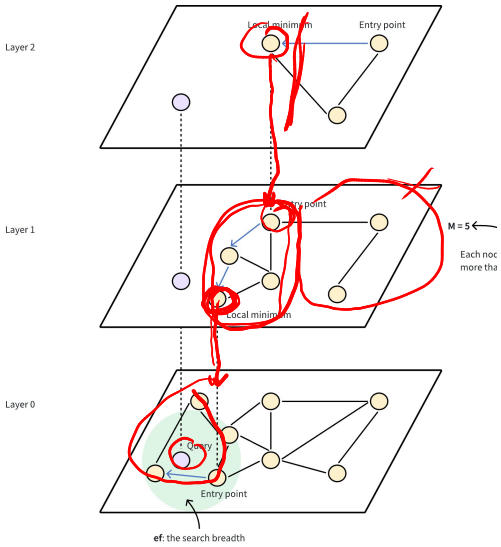


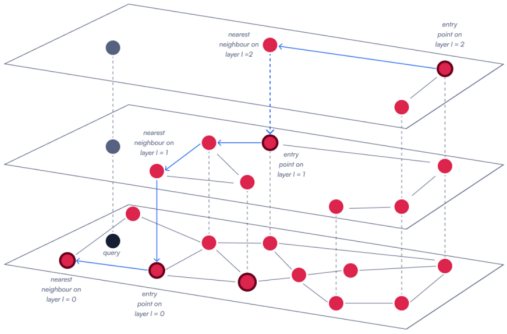





Layer=2
Layer=1
Layer=0
Decreasing characteristic radius
every vector
high
medium
Low

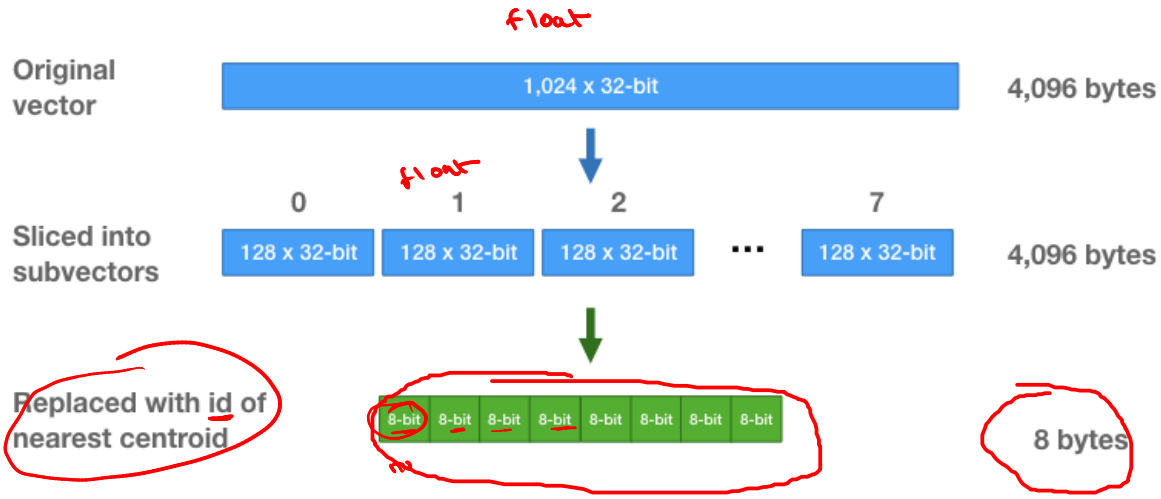

r=0

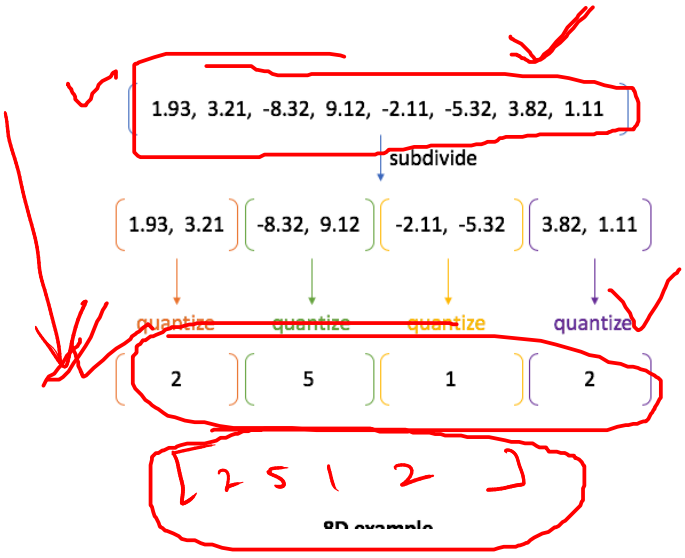

Layer 2
Layer 1
Layer 0
Local minimum
Entry point
Local minimum
Query
Entry point
ef: the search breadth
M = 5
Each noc
more tha

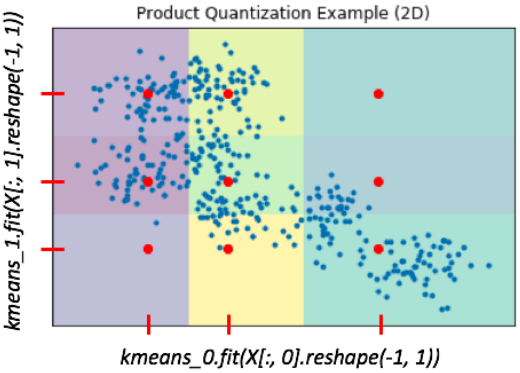

layer 2
layer 1
layer 0
nearest neighbour on layer l = 2
entry point on layer l = 2
nearest neighbour on layer l = 1
entry point on layer l = 1
query
nearest neighbour on layer l = 0
entry point on layer l = 0

Product Quantization (PQ)

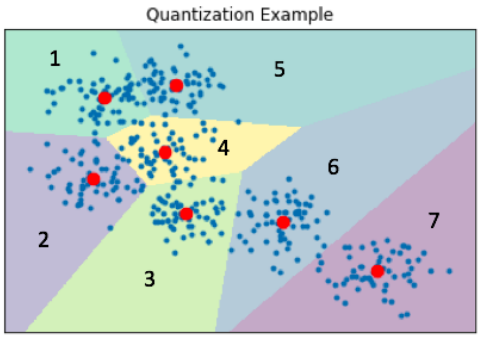
Compresses vectors into smaller, lower-dimensional representations (codes) to save memory. Distance calculations are performed on these compressed codes.

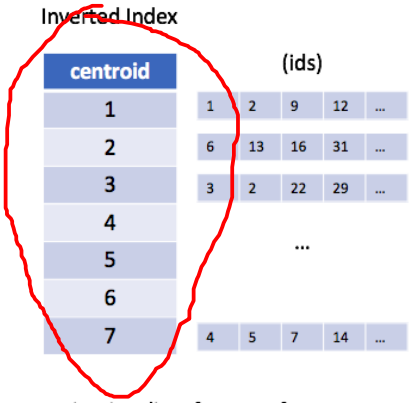




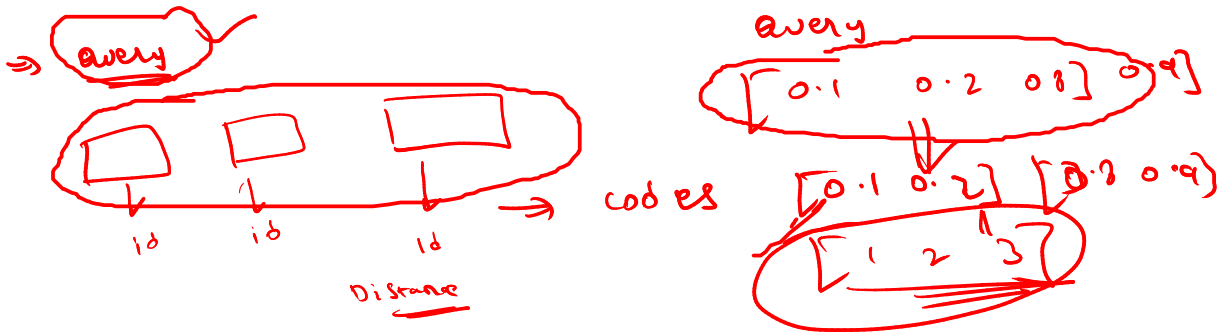


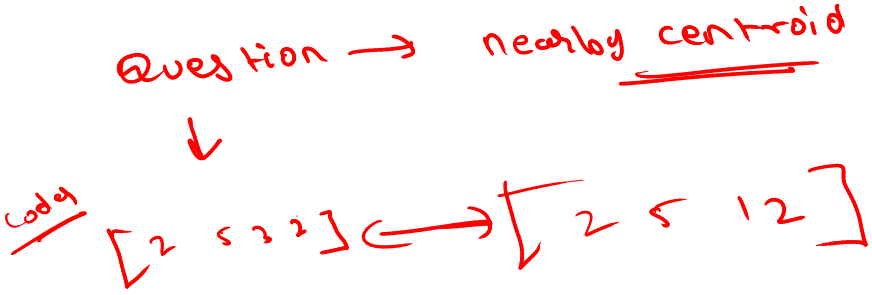
$X = (x_0, x_1)$





Assuming each vector has an identifier, we can maintain a list of vectors for each Voronoi cell. This type of data structure is known as an **inverted index**.




Question → nearby centroid
code
[2 5 3 2] ⟶ [2 5 12]